DAILY JAVA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Java – Beautifully Broken

Java learns to be lazy on purpose, and Netty ships 16 security fixes without a single CVE number

Issue #9 | Tue, 13 Oct 2026 | ⏱️ ~39 min read | Level 3/5

☕ **Today's two stories**

1. **Lazy Constants are heading for a fourth preview.** JEP 547 became a Candidate last week, proposing to preview `LazyConstant` once more with no changes. It's the JDK's answer to the double-checked-locking singleton most codebases contain, with the JVM's constant-folding optimisations as a bonus. Here's the problem it solves, how it works inside, and what changed in each of its three previous rounds.
2. **Netty 4.2.19 and 4.1.139 fix 16 security issues, and none of them had a CVE number at release.** Netty is the networking layer under Spring WebFlux, gRPC and many other Java frameworks. Here's what Netty does, how request smuggling and the other bug classes work, why scanners might not flag these yet, and why 4.1 has less than nine months left.
3. **On the radar:** 6 more things that happened, listed at the end.

STORY 1 Lazy Constants: the JDK takes on your double-checked locking

JDK · CORE-LIBS · JEP 547 (CANDIDATE) · PROMOTED FROM DRAFT, WEEK OF 5 OCT 2026 · ~17 MIN READ

Somewhere in your codebase there's a field that starts as `null` and gets filled in the first time someone asks for it. It might be a logger, a parsed config file, an expensive client or a lookup table. Around it sits a little ritual: a `volatile` keyword, a `synchronized` block, a second `null` check, and a comment saying "don't touch this." Java has never had a built-in way to say "compute this once, later, safely, and then treat it as a constant." For three releases, the JDK team has been previewing one. Last week, they proposed a fourth round, and this time they don't want to change anything.

WHAT HAPPENED

In the week of 5 October 2026, JEP draft 8389764 was promoted to Candidate status as **JEP 547: Lazy Constants (Fourth Preview)**, InfoQ reported in its roundup published on 12 October.[1] The JEP proposes a fourth preview "with no changes" from the previous round.[1]

A JEP ≈ *JDK Enhancement Proposal: the document that describes a significant change to Java and tracks it from draft to delivery* at **Candidate** status has been accepted into the OpenJDK backlog but is not yet aimed at a specific release. The next steps are *Proposed to Target* (a release is named and a short review starts) and *Targeted*. With JDK 27 already shipped in September, the next release a new preview could land in is JDK 28, due in March 2027, but JEP 547 hasn't been proposed for it yet.

This is the API's fourth round. It previewed in JDK 25 under the name **Stable Values** (JEP 502), then was renamed and slimmed down as **Lazy Constants** in JDK 26 (JEP 526), then revised again in JDK 27 (JEP 531).[1,2,3] InfoQ notes that the idea was also once called *Computed Constants*.[1] A preview that repeats with no changes is usually a sign that the design has settled and the team wants more real-world feedback before making it permanent.

The whole API is one interface and three factory methods on collections:

```
// JDK 27, preview: javac --release 27 --enable-preview
private static final LazyConstant<Logger> LOG =
        LazyConstant.of(() -> Logger.create(MyService.class));

LOG.get().info("started");  // computed on first get(), then constant
```

THE BACKGROUND YOU NEED

Why "lazy" matters at all

A Java application does a lot of work before it handles its first request: loading classes, running static initialisers and building objects. Every piece of that work you can defer, or skip because nobody needs it, makes startup faster. That's the idea behind **lazy initialisation** ≈ *computing a value the first time it's needed instead of up front*.

The cheapest way to hold a value in Java is a `static final` field. The **JIT compiler** ≈ *the part of the JVM that compiles hot bytecode into optimised machine code while the program runs* knows a `static final` field can never change after its class is initialised, so it can treat the value as a literal in compiled code. This is **constant folding** ≈ *an optimisation where the compiler replaces a read of a value that can never change with the value itself, and then simplifies the code around it*. If `DEBUG` is a `static final boolean` set to `false`, the JIT can delete every `if (DEBUG)` block.

The problem is timing. A `static final` field is set when its class is initialised, and you don't control exactly when that happens. If the value is expensive, you pay for it early, whether you need it or not. So developers reach for a non-final field that starts empty, and the JIT loses the ability to fold it, because a non-final field could change at any time.

The three workarounds every Java developer has used

1. The holder-class idiom. You put the value in a `static final` field of a tiny nested class. Java initialises a class only when it's first used (JLS §12.4), so the value is computed on first access, and the JVM's class-initialisation lock makes it thread-safe. It's lazy and fast, but it only works for `static` values, needs a class per value, and an exception during initialisation leaves the class permanently broken with a confusing `NoClassDefFoundError`.

2. Double-checked locking. You read a `volatile` field. If it's `null`, you lock, check again, and compute. It works for instance fields too, but it's easy to get wrong. Forget `volatile` and another thread can see a half-built object. It's also invisible to the JIT, because a `volatile` field can't be constant-folded.

3. A library helper. Many libraries ship a memoising wrapper. Spring's is `SingletonSupplier`, in `spring-core` since 5.1, and it's a good example of what these helpers do internally: a `volatile` field for the instance, a separate `initialized` flag, and a `ReentrantLock` around the slow path.[10] These helpers are correct and convenient, but to the JVM they're still ordinary objects with mutable fields.

Here are the two hand-written versions side by side. We compiled and ran both on JDK 21.

Two hand-rolled lazy values (JDK 21, both compile)

Holder-class idiom

```
class Config {
  static class Holder {
    static final Properties P =
        load();
  }
  static Properties props() {
    return Holder.P;
  }
}
```

Double-checked locking

```
static volatile Properties props;
static Properties props() {
  Properties p = props;
  if (p == null) {
    synchronized (Config.class) {
      p = props;
      if (p == null)
        props = p = load();
    }
  }
  return p;
}
```

Left: lazy and foldable, but static-only and one class per value. Right: works anywhere, but easy to break and invisible to the JIT.

The trick the JDK kept for itself

The JDK has had a way around this for years, but only for its own code. Inside `java.base` there's an internal annotation, `jdk.internal.vm.annotation.@Stable`. It tells the JIT that a field may start out `null` but, once it has a non-`null` value, will never change again. So the JIT can fold it after initialisation. The JDK's own implementation of lazy constants uses exactly this annotation.[6] Ordinary code can't use `@Stable`, because if you break its promise the JIT will happily keep using a stale value. Lazy Constants are, in effect, a safe public wrapper around that capability.

From Stable Values to Lazy Constants



HOW IT WORKS

The API

You create a lazy constant from a **Supplier** ≈ *java.util.function.Supplier: a function that takes no arguments and returns a value*, which the JDK docs call the *computing function*. Calling `get()` runs the function the first time and returns the stored result on every call after that.[4] The Javadoc spells out the guarantees:[4]

- **At most once.** If several threads call `get()` at the same time, only one runs the computing function. The others block until it finishes.
- **Safe publication.** The computation **happens-before** ≈ *the Java Memory Model's guarantee that the effects of one action are visible to another* any read of the result, so other threads never see a half-built object.
- **Never `null`.** If the function returns `null`, `get()` fails. If you need "maybe no value", store an `Optional`.
- **No surprises from `equals`.** Two lazy constants are equal only if they're the same object, and `equals`, `hashCode` and `toString` never trigger the computation.

There are also lazy collections, for when you have many values rather than one.[4,5]

- `List.ofLazy(size, i -> ...)` gives a fixed-size list where each element is computed on first access.
- `Map.ofLazy(keys, k -> ...)` gives a map with a fixed key set, where each value is computed on first access.
- `Set.ofLazy(candidates, e -> ...)`, new in JDK 27, gives a set where each candidate's membership is decided on first `contains`.

The JDK's own example for the lazy list is a small pool of components, created only when a thread first asks for one:[5]

```
static final List<OrderController> ORDERS =
        List.ofLazy(POOL_SIZE, _ -> new OrderController());

static OrderController orders() {
    long index = Thread.currentThread().threadId() % POOL_SIZE;
    return ORDERS.get((int) index);
}
```

The `_` is an **unnamed variable** ≈ *a Java 22+ feature: an underscore stands in for a parameter you don't use*. So with `POOL_SIZE` set to 3, at most three controllers are ever created, and only the ones actually requested.

What happens inside `get()`

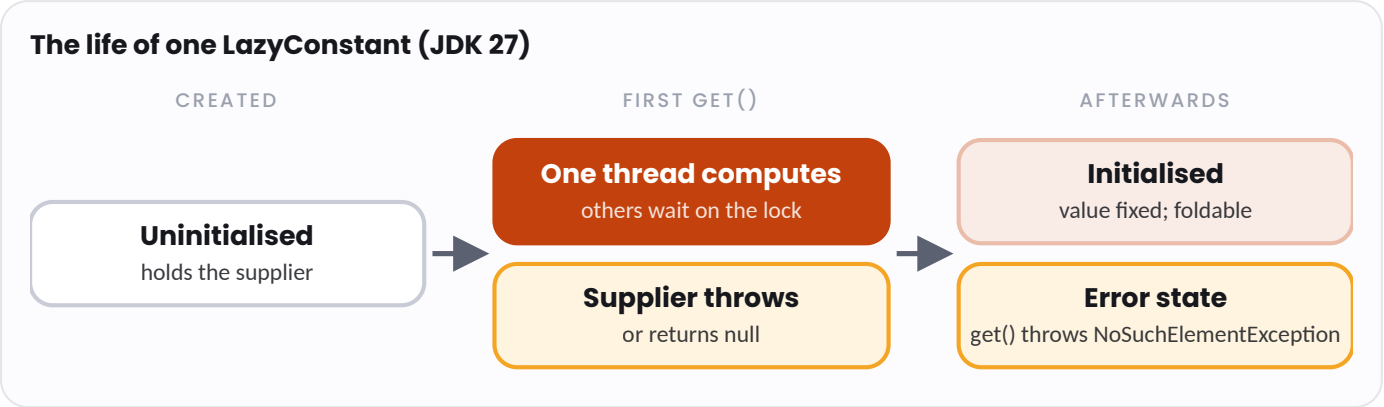
We read the implementation, `jdk.internal.lang.LazyConstantImpl`, in the JDK 27 source.[6] It's about as small as a thread-safe lazy value can be, and it shows why some of the API's rules exist.

The object has two fields. `constant` holds the value. It's annotated `@Stable`, and `null` means "not set yet", which is why the API refuses `null` results: there's no other way to tell "computed as `null`" from "not

computed". The second field, `computingFunctionOrExceptionType`, is `volatile` and holds the supplier until it has run.

The **fast path** is a single read of `constant` with *acquire* semantics. If it's non-`null`, that's the answer. This method is marked to always be inlined, so in hot code it costs about the same as reading a field. If the lazy constant is reachable from a `static final` field, the JIT can go further and fold it to the value itself.

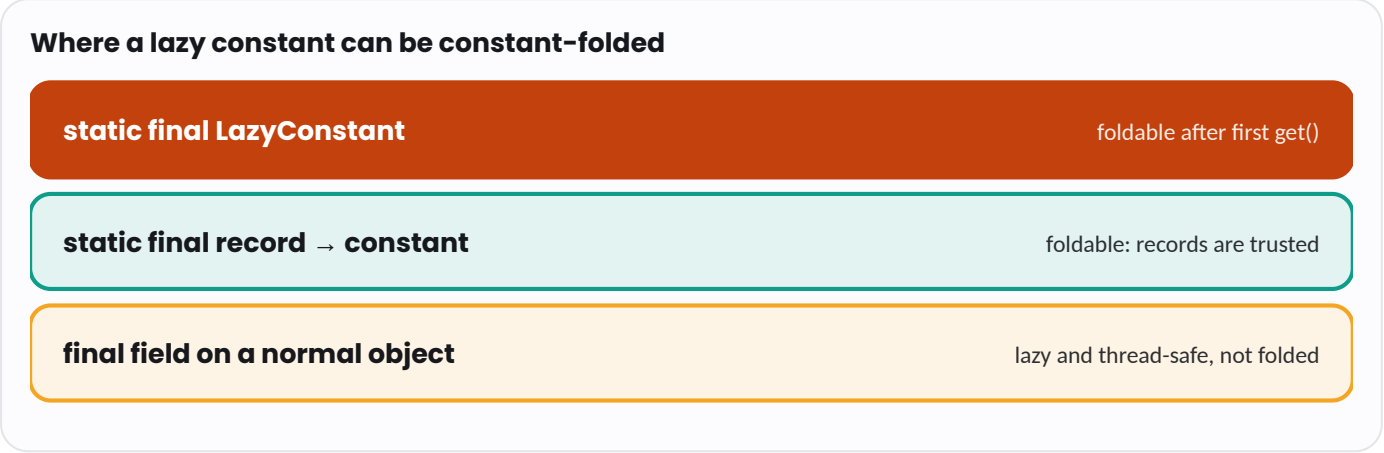
The **slow path** only runs while the value is missing. It takes a `synchronized` lock on the lazy constant, checks again, runs the supplier, stores the result with *release* semantics, and then sets the supplier field to `null` so the lambda (and anything it captured) can be garbage-collected. If the supplier throws, it stores the exception class's **name** in that field, because, as the source explains, holding the class itself would keep its **class loader** ≈ *the object that loaded a set of classes; while it's reachable, none of those classes can be unloaded* alive.[6]



When does the JIT actually fold it?

This decides whether you get the speed-up or just the convenience. The Javadoc says constant folding is possible only "if there is a direct reference from a `static final` field to a lazy constant", or a chain to it through *trusted fields*: `static final` fields, **record** ≈ *a Java 16+ class declared with `record`, whose fields are final and which the JVM trusts not to change* fields, or final instance fields in **hidden classes** ≈ *classes generated at runtime by frameworks via the Lookup API, not visible by name*.[4]

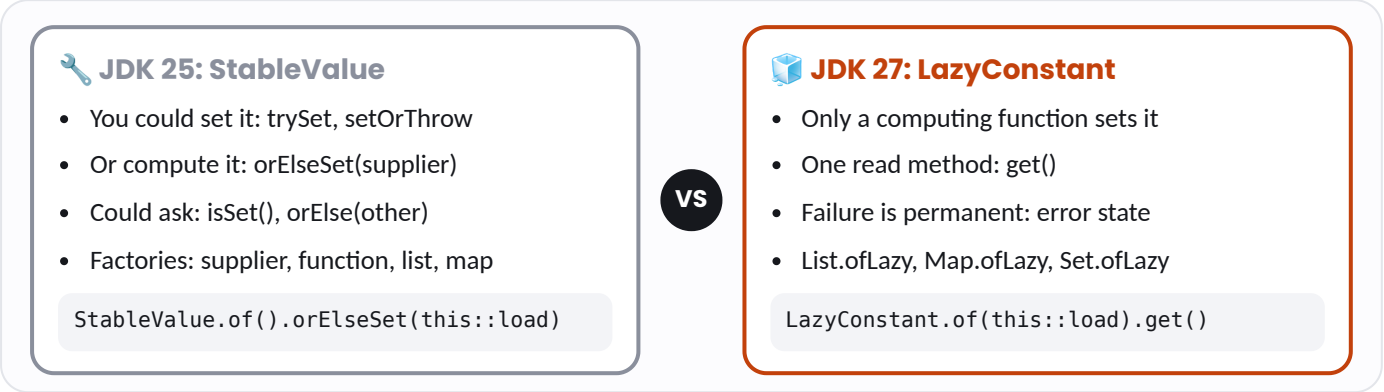
So `private static final LazyConstant<Logger> LOG` can be folded. A `private final LazyConstant<Logger> logger` on an ordinary object gives you laziness and thread safety, but the JIT still reads through it each time, because ordinary final instance fields aren't on the Javadoc's list of trusted fields. That's still a cheap read, but it isn't a constant.



What changed in each round

The three previous previews changed the API substantially, which is why a "no changes" fourth round matters. We compared the source of `StableValue` in JDK 25 and `LazyConstant` in JDK 26 and 27.[8,7,4]

- **JDK 25, Stable Values (JEP 502).** A low-level holder you could set yourself: `trySet`, `setOrThrow`, `orElseSet`, `orElse`, `orElseThrow` and `isSet`, plus factories for memoised suppliers, functions, lists and maps.[8]
- **JDK 26, Lazy Constants (JEP 526).** Renamed and cut down to factory-based initialisation only. The set-it-yourself methods were dropped. InfoQ quotes the JEP calling them "difficult to justify." `null` was banned, and the collection factories moved to `List.ofLazy` and `Map.ofLazy`. `orElse` and `isInitialized` remained.[2,7]
- **JDK 27, third preview (JEP 531).** `orElse` and `isInitialized` were removed, leaving `get()` as the only way to read. Failure handling changed: in JDK 26, a supplier that threw left the constant uninitialised so the next `get()` tried again. In JDK 27, the constant moves to a permanent error state and every later `get()` throws `NoSuchElementException` without calling the supplier again. `LazyConstant.of(x)` now always returns a new object, even if `x` is already a lazy constant, and `Set.ofLazy` was added.[7,4,9]
- **Fourth preview, Candidate (JEP 547).** No changes proposed.[1]



The direction is clear: from a flexible concurrency primitive to a narrow tool with one job, which is much easier to optimise and much harder to misuse.

WHAT IT MEANS FOR YOU

You can try it today, on JDK 27. It's a **preview feature** *≈ a fully implemented feature shipped switched off, so developers can try it and give feedback before it becomes permanent; it may still change*, so you need `--enable-preview` at compile time (with `--release 27`) and at run time. We couldn't download a JDK 27 build in our sandbox, so the snippets here are taken from the JDK's own Javadoc and source rather than compiled by us. Don't ship preview APIs in production code or in libraries other people depend on: class files compiled with preview features only run on that exact JDK version, with the flag.

Where it fits in a Spring application. Spring beans are already lazily managed singletons, so you don't need this for anything the container creates. Lazy constants are for the code around the container: `static` loggers and formatters, parsed resource files, precomputed tables, expensive clients in utility classes, and libraries

that don't want to depend on Spring or Guava. The pattern to look for is a `volatile` field with a `null` check and a lock. Each one is a candidate.

> 🌍 **In the real world**
>
> Spring Framework's `SingletonSupplier` is a good picture of what lazy constants replace. Its `get()` reads a `volatile` field and, if it's `null`, takes a `ReentrantLock`, checks an `initialized` flag, calls the supplier (falling back to a default supplier if the first returns `null`), then stores the result.[10] It's correct, and it supports `null` results, which `LazyConstant` deliberately doesn't. It's also a page of careful code that the JVM can't optimise as a constant. That trade-off, flexibility versus foldability, is exactly what the JDK team chose differently.

Watch the failure semantics if you used JDK 26. The change in JDK 27 matters for real code. If your supplier does I/O, such as reading a file or calling a config server, one failure now poisons the constant until the process restarts. That's often what you want for genuine configuration errors. For something that can fail transiently, either handle the retry inside the supplier or don't use a lazy constant for it.

Give feedback if you care. Collecting feedback is what a preview is for, and a fourth round with no changes may be the last good moment to tell the JDK team that something's wrong before the API becomes permanent.

> 💥 **Watch out for**
>
> - **It's still a preview.** JEP 547 is a Candidate, not targeted to any release yet. The API could still change or, in principle, be withdrawn.
> - **No `null`.** A supplier that returns `null` puts the constant in its error state. Wrap optional values in `Optional`.
> - **Failure is permanent (since JDK 27).** No retry, no reset. Later calls throw `NoSuchElementException` with the original exception's class name in the message.
> - **Folding needs `static final`** (or a trusted chain). An instance field gets laziness only.
> - **It never lets go.** A lazy constant strongly references its value forever, so caching something big in a long-lived object is a memory leak by design. Use a real cache if values should expire.
> - **Don't synchronise on it, and don't serialise it.** The implementation may lock on itself, and the type isn't `Serializable`.[4]

WHAT HAPPENS NEXT

The next step is for JEP 547 to be *Proposed to Target* a release, most likely JDK 28, with a short review period on the `jdk-dev` mailing list. If nothing changes in that round, the following one could make the API final. Lazy constants also fit the JDK's wider push on startup time: the implementation class carries an internal annotation, `@AOTSafeClassInitializer`,[6] which by its name marks it as safe to initialise ahead of time for the **AOT cache** ≈ *Project Leyden's ahead-of-time cache, which stores work done in a training run so later runs start faster* we covered in issue #2. Deferring work you might not need and caching work you always need are two halves of the same plan.

📖 Words from this story

JEP: JDK Enhancement Proposal, the document that describes a significant change to Java and tracks it from draft to delivery.

Lazy initialisation: computing a value the first time it's needed instead of up front.

JIT compiler: the part of the JVM that compiles hot bytecode into optimised machine code while the program runs.

Constant folding: replacing a read of a value that can never change with the value itself, then simplifying the code around it.

Supplier: `java.util.function.Supplier`, a function that takes no arguments and returns a value.

Happens-before: the Java Memory Model's guarantee that the effects of one action are visible to another.

Unnamed variable: Java 22+; `_` stands in for a parameter you don't use.

Class loader: the object that loaded a set of classes; while it's reachable, none of those classes can be unloaded.

Record: a Java 16+ class declared with `record`, whose fields are final and which the JVM trusts not to change.

Hidden classes: classes generated at runtime by frameworks via the Lookup API, not visible by name.

Preview feature: a fully implemented feature shipped switched off, for feedback before it becomes permanent.

AOT cache: Project Leyden's ahead-of-time cache of work done in a training run, so later runs start faster.

STORY 2

Netty's 16 silent fixes, and the clock on 4.1

ECOSYSTEM · SECURITY · NETTY 4.2.19.FINAL AND 4.1.139.FINAL · RELEASED 6 OCT 2026 · ~21 MIN READ

Open your `pom.xml` and search for "netty". There's a good chance you won't find it. Now run `mvn dependency:tree` on a Spring WebFlux service, a gRPC client or a Spark job, and it'll probably turn up several times. Netty is the networking code that much of the Java world runs on without ever choosing it. Last Tuesday, its maintainers shipped fixes for 16 security problems in one release. Not one of them had a CVE number, which means some of the tools you rely on to spot vulnerable dependencies may not flag them yet.

WHAT HAPPENED

On 6 October 2026, the Netty project released **Netty 4.2.19.Final** and **Netty 4.1.139.Final**, both described as "a bug-fix and security release."[11,12] The 4.2 release lists 16 security advisories, and the maintainers "strongly recommend upgrading." The 4.1 release lists 13 of them. The three missing from 4.1 are in modules that only exist in 4.2: the HTTP/3 codec and the two **io_uring** *≈ a modern Linux interface for asynchronous I/O, which Netty 4.2 supports as a first-class transport* transport modules.[11,12]

Each advisory links to a GitHub Security Advisory with a `GHSA-` identifier, but every CVE slot in the release notes reads `CVE-2026-XXXXX`. The maintainers explain:[11]

> "Note that due to overwhelming strain on the CVE infrastructure, we have not gotten a single CVE number assigned to these reports in time for our release. The advisories will be published without."

That's not new for Netty this year. The previous release, 4.2.18.Final on 9 September, carried the same note, with 30 advisories.[13] The same day, the project announced that **Netty 4.1 will reach end of life on 1 July 2027**, after which "we will no longer support Netty 4.1, and we will make no further releases in this series."[14]

The release also contains one deliberate breaking change. `FileUpload.setContentType` now throws `IllegalArgumentException` if you pass something that isn't a plausible **MIME type** *≈ a label like `text/plain` or `image/png` that says what kind of data a body or file contains*. Previously, it accepted anything.[11]

Downstream projects moved quickly. Apple's ServiceTalk merged its upgrade the same day, describing 4.2.19 as "a security and bug-fix release."[18] Apache Spark's upgrade pull request counts "16 security fixes."[19] Micronaut 5.2.2 shipped on 7 October with Netty 4.2.19.Final, InfoQ reported.[1] Spring Boot's maintenance branches for 4.0.x and 4.1.x and its `main` branch already manage Netty 4.2.19.Final, but the latest released versions, 4.0.8 and 4.1.1 (20 August), still ship 4.2.17.Final. Spring Boot 3.5.16 (25 June) ships 4.1.135.Final.[20]

THE BACKGROUND YOU NEED

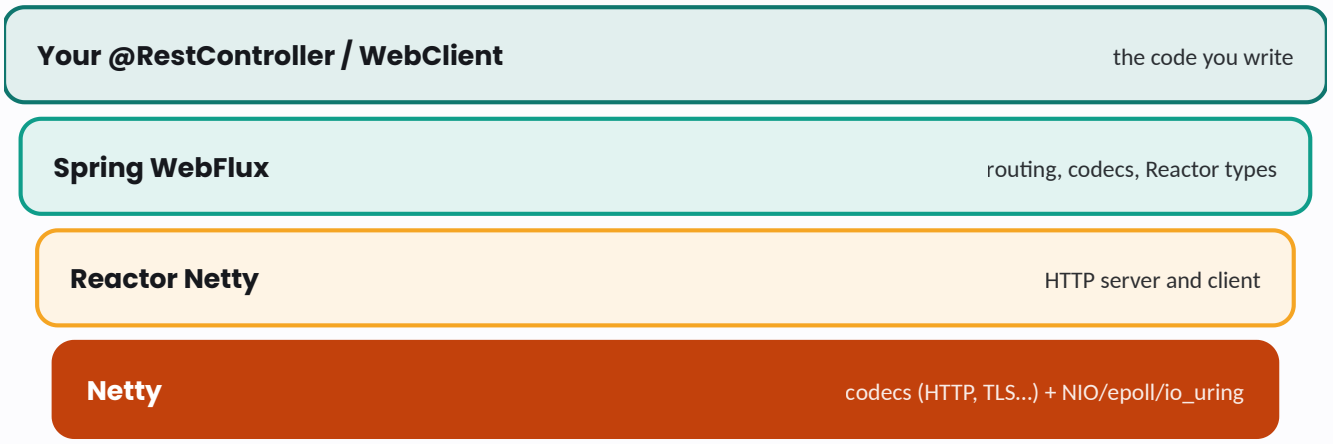
What Netty is

Netty *≈ an open-source Java framework for building network clients and servers on non-blocking I/O* handles the low-level work of network programming: opening sockets, reading and writing bytes without blocking threads, and

turning raw bytes into protocol messages and back. You give it a **channel pipeline** *≈ Netty's chain of handlers that each inbound or outbound message passes through, one step at a time* : one handler decrypts TLS, the next parses HTTP, the next decompresses, and your code sits at the end. The parsers are called **codecs** *≈ encoder/decoder components that convert between bytes on the wire and structured messages* , and Netty ships dozens of them, for HTTP/1.1, HTTP/2, HTTP/3, DNS, SOCKS, MQTT, the HAProxy PROXY protocol and more.

You rarely use Netty directly. Spring Boot's WebFlux starter is literally described as "Starter for using WebFlux and Reactor Netty",[21] and **Reactor Netty** *≈ the Reactor project's reactive HTTP/TCP client and server, built on Netty; WebFlux's default server* is the server and HTTP client it brings in. Netty's own list of adopters includes Google's gRPC for Java, Elasticsearch, Apache Spark, Netflix's Zuul gateway and Apple, whose engineers have given talks about running Netty at very large scale.[23]

Where Netty sits in a typical reactive Spring service



That's why a Netty advisory matters even if you've never typed `io.netty`. The bytes an attacker sends to your service are parsed by Netty before your code sees anything.

Two release lines, ten years apart

Netty 4.1.0.Final came out on 26 May 2016. Netty 4.2.0.Final followed on 3 April 2025, and since then feature work has gone into 4.2, with 4.1 getting only backported bug and security fixes.[14] Spring Boot 4.x manages 4.2, but Boot 3.5 still manages 4.1.[20] Anything still on 4.1 now has a deadline.

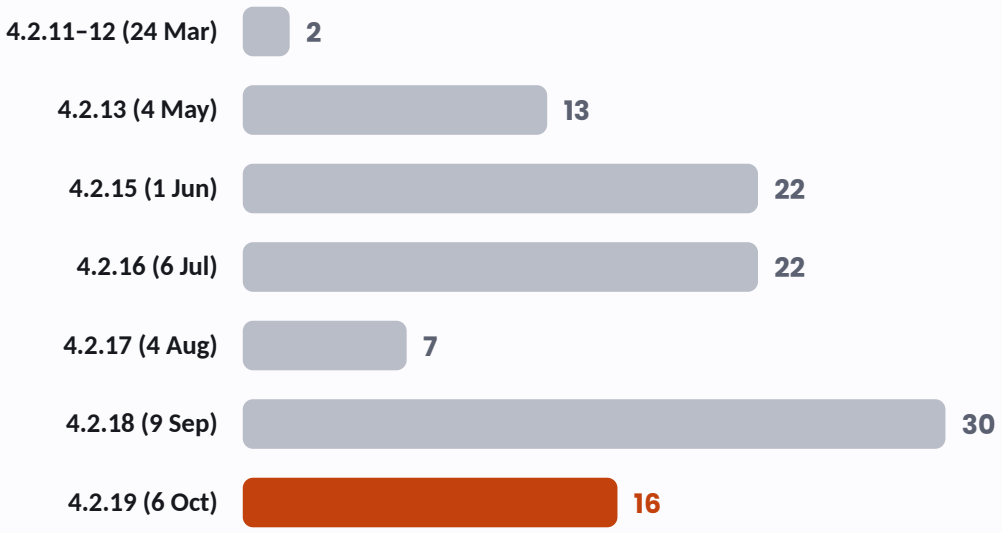
Netty's two lines



Why 2026 has been so busy

Counting the advisories linked from each 4.2 release announcement on netty.io gives a stark picture. In 2025, Netty's announcements linked 7 distinct advisories in total. In 2026 so far, they've linked 112.[15]

Security advisories per Netty 4.2 release, 2026



Our count of distinct GHSA links in each release announcement in the netty-website repository (read 13 Oct 2026). 4.2.11 and 4.2.12 were released the same day with the same two advisories. 4.2.14 listed none. All of 2025: 7.

Netty's notes don't say why. The timing matches what's happening across open source. A Cloud Security Alliance paper from May counted 48,185 CVEs published in 2025, 263% more than in 2020, with the first quarter of 2026 running about a third higher again, and it calls AI-assisted vulnerability discovery "the accelerant that has converted a manageable backlog into a structural failure."[25] The same paper reports that NIST, which runs the **NVD** ≈ *the US National Vulnerability Database, which adds severity scores and product data to published CVEs*, now plans full analysis for only 15–20% of new CVEs.[25] A **CVE** ≈ *Common Vulnerabilities and Exposures: the global ID scheme for publicly known vulnerabilities, e.g. CVE-2026-42580* ID is assigned by a **CNA** ≈ *CVE Numbering Authority: an organisation allowed to assign CVE IDs, such as GitHub for many open-source projects*. When those organisations are overloaded, a project like Netty can either wait or publish without the number. Netty chose to publish.

HOW IT WORKS

The 16 advisories fall into a handful of bug classes, and the fixes are all public in Netty's Git history. Reading them is a good lesson in how network code goes wrong. Netty's announcement gives each advisory a type and a module.[11] The detailed descriptions below come from the fix commits in the release,[16] matched to advisories by module and type. The advisory pages themselves weren't reachable from our sandbox, so treat that matching as ours.

Request smuggling: when two parsers disagree

Two advisories are in Netty's HTTP/1.1 codec: one labelled "request/response smuggling" and one "improper CRLF neutralization (request/response smuggling)."[11] **HTTP request smuggling** ≈ *an attack where a front-end proxy*

and a back-end server disagree about where one HTTP request ends, letting an attacker hide a second request inside the first is one of the most dangerous kinds of bug a web server's parser can have.

Most production services sit behind something: a load balancer, an API gateway or a CDN. The proxy and the backend both parse the same stream of bytes, and they have to agree on exactly where each request ends. HTTP/1.1 gives two ways to say how long a body is. `Content-Length: 48` says "48 bytes". `Transfer-Encoding: chunked` says "the body comes in pieces, each prefixed with its size in hex, ending with a zero-size piece." If a request is ambiguous (both headers, two different `Content-Length` values, a malformed chunk size) and the two parsers resolve it differently, the bytes one thinks are body, the other thinks are a new request.



The hidden request skips whatever the proxy enforces, such as authentication, path rules or rate limits. Depending on the setup, it can also be attached to the next user's connection.

The fix in this release for the plain smuggling advisory is a one-line change with a long history. HTTP has always forbidden conflicting `Content-Length` headers, and Netty already rejected them for HTTP/1.1. But its check had an exception: `if (multipleContentLengths && !isHttp10OrEarlier)`. Requests that claimed to be HTTP/1.0 skipped the validation. The commit, by Violeta Georgieva, deletes the exception so that multiple `Content-Length` values are "handled consistently regardless of the protocol version."[16]

The CRLF advisory is about the other direction. **CRLF injection** *≈ smuggling the carriage-return/line-feed characters that end an HTTP header line into a header value, so the receiver sees extra headers or a new message* happens when untrusted text ends up in a header without being checked. In a **multipart** *≈ the HTTP body format used for file uploads, where each part has its own small set of headers* upload, each part has its own `Content-Type` header. Netty's `FileUpload.setContentType` copied whatever string you gave it into that header. If that string came from user input and contained `\r\n`, the encoded request could carry injected headers. The fix validates the value against the MIME grammar and throws if it doesn't match. That's the breaking change mentioned above.[16]

Smuggling has been a theme all year. In May, Netty 4.2.13 fixed CVE-2026-42580, where the chunk-size parser built a number digit by digit in a 32-bit `int` and only checked whether the result went negative.[17] The advisory's proof of concept sends the chunk size `100000004` in hex. We checked the arithmetic on JDK 21: `(int) 0x100000004L` is `4`. So Netty read 4 bytes of body and parsed everything after them as a fresh request.

A classic Java trap: signed bytes

The SOCKS fix is worth reading if you write any binary-protocol code, because it's a mistake Java makes very easy. In **SOCKS5** *≈ a proxy protocol; a client's first message lists how many authentication methods it supports, then the*

methods, the client's greeting contains `NMETHODS`, a single byte from 1 to 255. Netty read it with `readByte()`, which returns a Java `byte`, and Java bytes are **signed**. Any value from 128 to 255 came out negative, so the decoder's `authSchemeNum > 0` check failed. It then took the "no methods" path without consuming the method bytes, and removed itself from the pipeline. The commit message explains that this forwarded "unconsumed, attacker-controlled bytes" to the next handler, which "can desync the pipeline or be smuggled in as a bogus follow-up protocol message."[16]

The SOCKS5 NMETHODS bug (we ran the arithmetic on JDK 21)

Before: signed read

```
// wire byte 0x80 = 128 methods
int n = in.readByte();
// n == -128
if (n > 0) {   // false!
  // read n method bytes
} else {
  // "no methods" path:
  // 128 bytes left unread
}
```

After: unsigned read

```
int n = in.readUnsignedByte();
// n == 128
if (n == 0) {
  // RFC 1928: must be >= 1
  // reject the greeting
}
// read all n method bytes
```

Simplified from Netty commit ef3a3e1 (SocksInitRequestDecoder). byte b = (byte) 0x80 gives -128; b & 0xFF gives 128.

Death by a thousand bytes

Six of the 16 advisories are "unbounded resource consumption," in the HTTP/1.1, HTTP/2, HTTP/3, DNS, XML and base codec modules.[11] These are **denial of service** *≈ an attack that makes a service unavailable, here by making it burn CPU or memory, rather than by stealing data* bugs, and the fix commits show a recurring pattern: work that grows faster than the input.

The clearest example is `DelimiterBasedFrameDecoder`, a general-purpose codec that splits a byte stream on a delimiter (often a zero byte). Every time new bytes arrived, it re-scanned the entire buffered input from the start. An attacker who sends bytes with no delimiter, one per TCP packet, forces a full rescan per byte. That's **quadratic** *≈ work that grows with the square of the input: 10× the data means 100× the CPU* CPU work, which the commit says "can pin an event-loop thread with just a few trickle connections."[16] Pinning an **event loop** *≈ the small set of threads in Netty that handle I/O for many connections each; if one is busy, every connection on it stalls* thread is a big deal in Netty, because each one serves many connections. The fix remembers where the last scan stopped, the way Netty's line-based decoder already did. Similar commits bound memory in the HTTP/2 header-compression table handling, cap a DNS name-decoding buffer, and stop the XML frame decoder rescanning unbalanced elements.[16]

Security controls that failed open

The most uncomfortable bugs are the ones where a security feature quietly does nothing.

- **IP filtering ("improper access control", netty-handler).** `IpSubnetFilter` lets you allow or deny connections by address range. If you configured `ACCEPT 10.0.0.0/8` plus a more specific `REJECT 10.5.0.0/16`, its de-duplication step dropped the REJECT rule as "covered" by the larger range, so every address in it was let in. A related bug meant an exception in your own `accept()` logic left the channel open instead of closing it. Overlapping rules of different types are now rejected at configuration time, and errors close the connection.[16]
- **SNI routing ("SNI routing bypass", netty-handler).** SNI ≈ *Server Name Indication: the hostname a TLS client sends first, so one server can pick the right certificate and settings per domain* lets one Netty server choose a TLS configuration per hostname. Empty TLS handshake records, which the TLS 1.3 spec forbids, could slip past the check and also make Netty buffer "an unbounded number" of them. Netty now fails the handshake.[16]
- **Certificate revocation ("time-of-check/time-of-use", netty-handler-ssl-ocsp).** OCSP ≈ *Online Certificate Status Protocol: a way to ask a certificate authority whether a certificate has been revoked* validation could fail because a peer's certificate was revoked, yet bytes that peer had already sent were still passed on to the next handler. They're now discarded.[16]
- **CORS caching ("origin validation error", netty-codec-http).** For the special `null` origin, Netty's `CorsHandler` set `Access-Control-Allow-Origin: null` but forgot `Vary: Origin`, the header that tells shared caches the response depends on the caller's origin. A CDN could then serve that response to unrelated sites.[16]

WHAT IT MEANS FOR YOU

1. Find out which Netty you actually run. Netty is almost always a **transitive dependency** ≈ *a library you get because one of your dependencies needs it, without declaring it yourself*, so ask your build tool rather than your `pom.xml`: `mvn dependency:tree -Dincludes=io.netty` or `./gradlew dependencyInsight --dependency io.netty`. Look for two things: whether you're on 4.1 or 4.2, and whether every `io.netty` artifact has the same version. Netty's migration guide warns that mixing versions, especially via shaded copies that hide Netty inside another JAR, is a common source of trouble.[24]

2. Upgrade now if you terminate HTTP with Netty. If your service runs on WebFlux, Micronaut, gRPC or anything else where Netty parses requests from the internet or from a proxy, the smuggling and resource-exhaustion fixes apply to you. On Spring Boot you don't have to wait for the next Boot patch release. Boot manages Netty's version through a property, so you can override it: set `<netty.version>4.2.19.Final</netty.version>` in your Maven `<properties>`, or `extra["netty.version"] = "4.2.19.Final"` with the Gradle dependency-management plugin, the same pattern Boot's documentation shows for other managed libraries.[22] Run your tests, because of the `setContentType` change. When the next Boot patch release arrives with 4.2.19 managed, remove the override.

3. Don't rely on CVE-only tooling for this one. Tools that match dependencies against CVE IDs alone may not flag these advisories until IDs are assigned. Tools that read GitHub's advisory database work from GHSA IDs directly. If your organisation's patch policy is triggered by "a CVE with CVSS ≥ 7", note that these 16 issues have no CVE yet, and Netty's announcement doesn't give severity scores either.[11] Someone has to make the call by reading the advisories.

4. If you're on 4.1, start the 4.2 migration. Spring Boot 3.5 applications are on 4.1 today.[20] After 1 July 2027 there will be no more 4.1 releases at all, including security fixes, and judging by this year, there will be fixes to miss.[14] Netty's migration guide recommends a staged approach.[24]

🧪 The Netty 4.2 migration checklist, from Netty's own guide[24]

- First get onto the latest 4.1.x and make it work.
- **Configure TLS hostname verification explicitly** in every client, using `SslContextBuilder.endpointIdentificationAlgorithm(...)`. Netty 4.2 turns it **on** by default (4.1 left it off), so clients that connect to hosts with mismatched certificates will start failing. This is the biggest risk for most systems.
- Pin the old memory allocator with `-Dio.netty.allocator.type=pooled`, because 4.2 makes the new `adaptive` allocator the default. Switch to `adaptive` later, after a soak test.
- Upgrade every component at once: 4.1 and 4.2 can't coexist on the classpath.
- If you used the incubator io_uring transport, move to `netty-transport-native-io_uring` (new package `io.netty.channel.uring`, Java 9+).

💥 Watch out for

- **The 4.1 release doesn't get everything.** 4.1.139 fixes 13 of the 16 issues. The HTTP/3 and io_uring fixes exist only in 4.2.19.[11,12]
- **Breaking change.** `FileUpload.setContentType("foo")` now throws `IllegalArgumentException`. Code that passes user-supplied or sloppy content types to Netty's multipart encoder needs a look.[11]
- **Your framework may pin its own Netty.** Gateways and SDKs sometimes shade or force a Netty version. Check the resolved tree, not just your BOM.
- **No severity scores published.** "Moderate" or "critical" isn't stated in the release notes. Don't assume these are minor because they're unscored.
- **Our mapping is inferred.** We matched fix commits to advisories by module and description. The authoritative details are in each GHSA advisory linked from the release notes.

WHAT HAPPENS NEXT

CVE IDs for this batch, and for September's 30, should follow once the CVE system catches up, at which point many more scanners will flag them. Expect your security tooling to "discover" these weeks after the fix shipped. Spring Boot's next patch releases for 4.0.x and 4.1.x will bring 4.2.19 automatically. Their branches already manage it,[20] and Spring's new monthly "Patch Thursday" release train, which we covered in issue #2, should make those patch releases more predictable. Netty 4.1 has until 1 July 2027.[14] If this year's pace continues, there may well be several more security releases before then, and every one is a reason to plan the 4.2 move now rather than in June.

📖 Words from this story

io_uring: a modern Linux interface for asynchronous I/O, which Netty 4.2 supports as a first-class transport.

MIME type: a label like `text/plain` or `image/png` that says what kind of data a body or file contains.

Netty: an open-source Java framework for building network clients and servers on non-blocking I/O.

Channel pipeline: Netty's chain of handlers that each inbound or outbound message passes through.

Codecs: encoder/decoder components that convert between bytes on the wire and structured messages.

Reactor Netty: the Reactor project's reactive HTTP/TCP client and server, built on Netty; WebFlux's default server.

NVD: the US National Vulnerability Database, which adds severity scores and product data to published CVEs.

CVE: Common Vulnerabilities and Exposures, the global ID scheme for publicly known vulnerabilities.

CNA: CVE Numbering Authority, an organisation allowed to assign CVE IDs.

HTTP request smuggling: a proxy and a backend disagree about where a request ends, letting an attacker hide a second request inside the first.

CRLF injection: sneaking line-ending characters into a header value so the receiver sees extra headers or a new message.

Multipart: the HTTP body format used for file uploads, where each part has its own headers.

SOCKS5: a proxy protocol whose greeting lists how many authentication methods the client supports, then the methods.

Denial of service: making a service unavailable by exhausting CPU or memory.

Quadratic: work that grows with the square of the input.

Event loop: one of the few Netty threads that each handle I/O for many connections.

SNI: Server Name Indication, the hostname a TLS client sends so the server can choose certificate and settings.

OCSP: Online Certificate Status Protocol, a way to check whether a certificate has been revoked.

Transitive dependency: a library you get because one of your dependencies needs it.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **Structured Concurrency is Proposed to Target JDK 28:** JEP 543 would finalise `StructuredTaskScope` unchanged after seven previews, with its review due to end today (13 Oct). See issue #5 for the full explainer.[1]
- **ZGC's self-sizing heap is Proposed to Target JDK 28:** JEPs 545 (faster startup and warmup) and 546 (adaptive heap sizing), from issue #8, have review periods ending 14 October.[1]
- **Apache Grails 8.0.0:** the Groovy web framework's new major requires JDK 21 and Gradle 9.7, and runs Grails Promises on Spring Boot's task executor.[1]
- **Open Liberty 26.0.0.10:** the Jakarta EE runtime adds JDK 27 support and fixes seven CVEs. It warns that G1 is now the default collector everywhere on JDK 27, which can change behaviour in small containers.[1]
- **Spock 2.5.0:** the Groovy-based testing framework adds Groovy 6 and Spring Framework 7 support, with some breaking changes.[1]
- **LangChain4j 1.22.0:** adds a Bedrock batch chat model and decision-model classes for the OpenAI Decisions API.[1]

1. **Java News Roundup: Grails 8, Open Liberty, GlassFish, Infinispan, LangChain4j, Spock**, Michael Redlich, InfoQ, 12 Oct 2026.
2. **JEP 526 Simplifies Deferred Initialization ahead of JDK 26**, A N M Bazlur Rahman, InfoQ, 2 Dec 2025. Candidate announcement: **New candidate JEP: 526**, Mark Reinhold, jdk-dev, 26 Sep 2025.
3. **JEP targeted to JDK 27: 531: Lazy Constants (3rd Preview)**, Inside.java, 17 May 2026; **New candidate JEP: 531**, Mark Reinhold, jdk-dev, 25 Feb 2026.

4. **java/lang/LazyConstant.java** and `java/util/List.java`, `Map.java` at tag `jdk-27-ga`, OpenJDK (read via git; identical on `master` as of 12 Oct 2026).
5. **java/util/Set.java and List.java** at tag `jdk-27-ga` (`Set.ofLazy`, `@since 27`; `List.ofLazy` example), OpenJDK.
6. **jdk/internal/lang/LazyConstantImpl.java** at tag `jdk-27-ga`, OpenJDK.
7. **java/lang/LazyConstant.java at jdk-26-ga**, OpenJDK; our diff against `jdk-27-ga`.
8. **java/lang/StableValue.java at jdk-25-ga**, OpenJDK.
9. **JEP 531 – Lazy Constants, Journey continues**, Sebastian Rabiej, SoftwareMill, 16 Mar 2026 (updated 15 Sep 2026).
10. **SingletonSupplier.java**, Spring Framework `main` (read 13 Oct 2026).
11. **Netty 4.2.19.Final released**, Chris Vest, netty.io, 6 Oct 2026.
12. **Netty 4.1.139.Final released**, Chris Vest, netty.io, 6 Oct 2026.
13. **Netty 4.2.18.Final released**, Chris Vest, netty.io, 9 Sep 2026.
14. **Netty 4.1 will be EOL on July 1st, 2027**, Chris Vest, netty.io, 9 Sep 2026.
15. **Netty news archive**, via the **netty-website repository** (our count of GHSA links per release post, 13 Oct 2026).
16. **Commits between netty-4.2.18.Final and netty-4.2.19.Final**, netty/netty on GitHub, notably 4029f07 (multiple Content-Length), faec63c (multipart Content-Type), ef3a3e1 (SOCKS NMETHODS), 85498e0 (DelimiterBasedFrameDecoder), d82c6c3 (IpSubnetFilter), 20f3aae (SNI), c3f7088 (OCSP), f4081c5 (CORS).
17. **Netty vulnerable to HTTP Request Smuggling due to incorrect chunk size parsing, CVE-2026-42580**, GitHub Advisory Database, published 5 May 2026.
18. **Upgrade Netty 4.2.18 -> 4.2.19**, apple/servicetalk PR #3680, 6 Oct 2026.
19. **SPARK-60022: Upgrade Netty to 4.2.19.Final**, apache/spark PR #59266.
20. **spring-boot-dependencies build.gradle**, Spring Boot, at tags v4.1.1, v4.0.8, v3.5.16 and branches main, 4.1.x, 4.0.x, 3.5.x (read via git, 13 Oct 2026).
21. **spring-boot-starter-webflux build.gradle**, Spring Boot 4.1.x.
22. **Managing dependencies: custom-version examples**, Spring Boot Gradle plugin documentation sources, 4.1.x.
23. **Adopters**, Netty project wiki.
24. **Netty 4.2 Migration Guide**, Netty project wiki.
25. **The NVD Infrastructure Crisis: AI Discovery Overwhelms Tracking**, Cloud Security Alliance Labs, 4 May 2026.